Encode, Generate, Review, Validate: Explicit Cognitive Modes with Persistent Goal State in a Shared Transformer

Paper 1 – Experimental Inception Draft

August 23, 2026

Abstract

Decoder-only language models use the same causal transformer stack for understanding a request, generating an answer, and implicitly evaluating what has been generated. We study a minimal alternative that preserves a shared transformer body but introduces explicit operating modes and persistent task state. The model first encodes the complete task with bidirectional self-attention, extracts a latent goal representation Z_G, generates causally while conditioned on this latched state, re-encodes the completed output in review mode, extracts a realized-output representation Z_Y, and validates the candidate by comparing intended and realized state. We define parameter-, data-, and compute-controlled baselines, machine-checkable multi-constraint generators, held-out near-miss validation, causal interventions on Z_G and mode identity, calibration metrics, and immutable run artifacts. An executable CPU-scale scaffold accompanies this inception draft; it validates the protocol but does not yet support empirical claims.

1 Problem Statement

A decoder-only transformer must encode a task and generate its solution under a causal mask. Even if a prompt is entirely available before answer generation, hidden representations at prompt position i are constrained to $x_{\leq i}$. We test whether separating encode, generate, review, and validate improves goal fidelity while retaining a shared neural substrate. The modes are experimental interventions, not claims of unique correspondence to human cognition.

2 Architecture

Let

$$H^{(m)} = F_\theta(X, Z, e_m, \mathcal{M}_m), \tag{1}$$

where e_m is a learned mode embedding and $\mathcal{M}_m$ a mode-specific attention mask. Encode/review use bidirectional masks; generation is causal. The first design keeps θ shared.

For an input of length n, the causal mask permits attention when $j \leq i$, whereas the bidirectional mask permits every non-padding pair (i, j). Mode embeddings remain present when masks are equal, so mask and mode identity can be ablated separately. All reported baseline models instantiate the same token, position, mode, transformer, goal, review, and validator parameters. We report both total and active parameters; inactive heads are retained to make checkpoint shapes and total parameter counts exactly comparable.

2.1 Encode and Goal Extraction

$$H_E = F_\theta(X, \varnothing, e_E, \mathcal{M}_{bi}), \qquad Z_G = Q_G(H_E). \tag{2}$$

Q_G operates before vocabulary projection. The executable V0 uses masked mean pooling followed by a learned projection. Subsequent registered variants compare a special goal token, one learned query, then $k \in \{4, 8\}$ facet queries. Shared causal/bidirectional mask operation has precedent [2]; the question here is its use inside a persistent goal-state cycle.

2.2 Generation

$$p(Y \mid X, Z_G) = \prod_{t=1}^{T} p_\theta(y_t \mid y_{<t}, X, Z_G, e_G). \tag{3}$$

The V0 conditioning is an additive broadcast projection $h'_{0,t} = h_{0,t} + W_Z Z_G$ applied to the generation input stream. Token-level encoded prompt states are passed into the causal generation pass, isolating bidirectional encoding without requiring a separate decoder. The current executable harness also supports a continuous-prefix diagnostic in which Z_G is projected into a small bank of learned latent prefix vectors prepended to the generation context, and a prefix-KV diagnostic in which separate goal-derived key and value memories are exposed at each self-attention layer without becoming query or residual-stream positions. These mechanisms are related to continuous prefix conditioning [3]. Lightweight cross-attention remains a registered follow-up. Goal-conditioned residual gating is deferred. The primary condition latches Z_G throughout generation.

The current executable next-iteration harness additionally supports configurable goal reinjection schedules during generation: input-only, early-layer, periodic, all-layer, and late-layer additive reinjection of the same latched latent condition, together with a continuous-prefix alternative that provides goal access through dedicated latent prefix vectors instead of additive state injection. These are diagnostic controls rather than new headline claims. They ask whether persistent goal influence depends mainly on what Z_G contains, on how often the generator can access it across residual depth, or on the form of the conditioning channel itself.

2.3 Review and Validation

For completed output Y,

$$H_R = F_\theta(Y, Z_G, e_R, \mathcal{M}_{bi}), \qquad Z_Y = Q_R(H_R). \tag{4}$$

Compare final causal state, an extra causal pass, bidirectional Y-only review, and bidirectional (X, Y) review. A validator computes

$$(r, \mathbf{r}_f, \Delta Z) = V_\phi(Z_G, Z_Y), \tag{5}$$

with scalar success probability, optional per-facet satisfaction, and discrepancy state. A baseline head uses

$$u = [Z_G; Z_Y; Z_G - Z_Y; Z_G \odot Z_Y], \quad r = \sigma(\mathrm{MLP}(u)). \tag{6}$$

3 Research Questions and Hypotheses

RQ1: Does bidirectional task encoding help under matched compute? RQ2: Does latched Z_G improve requirement retention, especially for long outputs? RQ3: Does bidirectional review improve

error detection beyond causal alternatives? RQ4: Does (Z_G, Z_Y) validation outperform simply rereading prompt and answer? RQ5: Can validation improve selection/retry under compute controls? RQ6: Do modes and Z have identifiable causal/mechanistic effects?

We expect encode gains for distributed/late constraints, Z gains to increase with generation length and simultaneous requirements, review gains for global inconsistencies/omissions, and intended-realized comparison to improve facet localization and candidate ranking.

4 Controlled Dataset Suite

4.1 Multi-Constraint Generation

Each prompt contains K independently checkable requirements. Vary K, prompt length, requirement position, distractor count, output length, and requirement type (content, order, format, exclusion, cross-part consistency). Store structured ground truth and deterministic validators.

4.2 Late Constraints and Distractors

Move decisive constraints to the end while holding semantics fixed. Add instruction-like non-authoritative distractors. These tasks test global encoding and whether Z captures task role rather than lexical salience.

4.3 Ordered and Deterministic Tasks

Require multiple ordered sections or transformations with independent checks. Include arithmetic, symbolic transformations, structured data, and small code tasks with executable tests.

4.4 Candidate Pairs

Construct valid/near-miss pairs differing in exactly one requirement for facet validation, ranking, and mechanistic probes.

4.5 Executable V0 Generator

The V0 generator represents each requirement as a separately checkable binary facet while varying facet count, order, filler length, and the number of explicitly delimited distractors. One authoritative requirement is always placed last. Train, validation, and test use both disjoint deterministic example namespaces and different prompt topologies: grouped requirements, reversed/grouped requirements, and locally interleaved authoritative/distractor blocks. The ordered target contains one token per active facet. Training corruptions flip one facet, validation targets the late facet, and test corruptions mix single and double flips, truncation, and invalid end markers. A counterfactual prompt and target flip the same authoritative facet, enabling a directed Z_G substitution test. Every run records the generator version, split family, and SHA-256 hash of the fully materialized prompts and labels. This synthetic vocabulary tests the architecture and harness, not general language understanding; natural templates, executable transformations, code tasks, and pretrained backbones are required in the next empirical stage.

5 Baselines and Ablations

ID	Encode/State	Review/Validation
B0	causal decoder, no Z	none
B1	B0 + matched residual MLP	none
B2	extra causal encode, no Z	none
B3	bidirectional encode, no Z	none
B4	causal encode + Z_G	none
B5	bidirectional encode + Z_G	none
B6	B5	final generation state
B7	B5	extra causal review
B8	B5	bidirectional Y review
B9	no persistent Z_G	bidirectional (X, Y) reread
B10	bidirectional encode + Z_G	bidirectional (X, Y), (Z_G, Z_Y)

All rows have the same instantiated parameter count. B1 activates a capacity-matching residual MLP without task state; B2 spends an extra causal pass; B3 and B5 pass token-level encoded prompt states into generation. Later work compares shared weights with small mode adapters and fully separate modules only if bidirectional/causal interference is observed.

6 Training

The executable training protocol supports an initial goal warmup followed by joint generation, goal decoding, validation, facet localization, and pairwise ranking. Warmup minimizes masked facet reconstruction and same-goal paraphrase invariance before Z_G is used as a generation condition. Joint checkpoints are selected only by loss on the held-out validation topology, with configurable evaluation cadence and bounded early stopping. The broader empirical progression remains: Stage 0 establishes a base LM or small pretrained model with exact disabled-feature parity. Stage 1 mixes causal/bidirectional mask training with explicit mode embeddings. Stage 2 adds goal objectives:

$$L = L_{LM} + \lambda_G L_G, \tag{7}$$

where L_G may include facet decoding, paraphrase-invariant contrastive learning, structured requirement reconstruction, or predictive usefulness. Stage 3 adds

$$L = L_{LM} + \lambda_G L_G + \lambda_V L_V + \lambda_R L_{rank}. \tag{8}$$

Use deterministic external labels where possible. Only after calibration should Stage 4 test retry/revision or limited RL.

For V0, L_G combines masked binary facet decoding with cosine paraphrase invariance, L_V is BCE on valid and deterministically corrupted candidates, and $L_{rank} = \log(1 + \exp(r^- - r^+))$. A separate multilabel BCE trains facet satisfaction. Loss weights, warmup duration, optimizer, seed, validation cadence, and gradient clipping are configuration values rather than hidden constants.

7 Inference Policies

Single generation is the base. Validation-only scores without changing output. Best-of-N selects $Y^* = \arg\max_i r_i$. Threshold retry regenerates if $r < \tau$. Revision conditions on discrepancy:

$$Y' = G(X, Z_G, Y, \Delta Z). \tag{9}$$

Every result reports model calls, generated tokens, approximate FLOPs, latency, and wall time.

8 Metrics

External task metrics: accuracy, exact match, F1, full-task pass rate, fraction of satisfied constraints, per-facet recall, executable-test pass rate, and degradation with generation length.

Validation: AUROC, AUPRC, Brier score, expected calibration error, ranking accuracy, best-of-N oracle gap, false-positive rate on confidently wrong outputs, and facet localization accuracy.

Goal representation: same-goal paraphrase similarity, different-goal separation, linear-probe facet recovery, Z stability, sensitivity to requirement position, and causal interventions.

Mechanistic: attention entropy/dilution; residual update norms; update/state alignment; refinement versus complementary/orthogonal update versus interference; layer/head similarity; mode-conditioned representation similarity; temporal feature stability.

9 Causal Intervention Suite

The executable evaluation zeroes Z_G; shuffles it across examples; substitutes a same-goal paraphrase state; substitutes a state from a prompt with one flipped facet; removes mode embeddings while retaining masks; and swaps encode/generate embeddings. Exact-match interventions use the same active-facet and required-end-token criterion as primary evaluation. Counterfactual evaluation separately reports whether the targeted facet changes, whether it changes while all untouched active facets remain correct, and whether the complete counterfactual target is exact; raw intervention generations are retained for audit. Registered extensions perturb learned facet directions, freeze/unfreeze Q_G, swap review identity, and compare latched with recomputed Z_G. A genuine goal representation should preserve behavior under same-goal substitution, degrade under zero/shuffle interventions, and alter the targeted output facet under counterfactual substitution. Correlation or probe accuracy alone is insufficient.

10 Compute and Parameter Controls

Mode architectures can win trivially by spending extra compute. Report training/inference FLOPs, forward-pass count, sequence lengths, parameter count, and latency. Compare review against equivalent causal compute; best-of-N validation against random and likelihood-based selection; and Z parameters against matched width/MLP additions. Distinguish quality-per-sample from quality-per-compute.

The harness counts actual shared-transformer calls and non-padding token positions at runtime. It reports base generation, validation, selection-time validation, and diagnostic interventions separately, together with instantiated and active parameter counts. The implementation-level estimate is $2P$ FLOPs per forward token-position and $6P$ per training token-position for P parameters; this coarse proxy is always accompanied by measured wall time and raw counts. It is not used as a substitute for profiler-based FLOPs in headline experiments. Prompt encoding and Z_G are computed once, cached, and latched for an autoregressive candidate; unit tests assert one encode call plus one causal call per generated token.

11 Expected Failure Modes

Shortcut validation: mitigate with held-out generators and adversarial near-misses.
Goal collapse: Z may encode prompt identity rather than intent; test paraphrases, probes, and substitutions.
Mode collapse: embeddings may be ignored; probe and swap modes.
Bidirectional/causal interference: shared weights may lose generation quality; only then test adapters/partial sharing.
Self-confirmation: generator and validator may share blind spots; use deterministic external validators.
Compute illusion: gains may vanish under matched budget; treat this as a valid negative result.

12 Future Goal Dynamics

If Z_G reliably represents explicit tasks and ΔZ unmet requirements, next introduce explicit task transitions and completion evidence. Future state can distinguish root, active, parent, pending, blocked, suspended, and completed goals. Only after known hierarchies work should the model learn decomposition

$$D(z_g, h) \to \{z_{g_1}, \ldots, z_{g_k}\}. \tag{10}$$

Thus Paper 1 is a substrate for planning rather than an attempt to solve planning.

13 Future Gated Residual Integration

Once Z is meaningful, test

$$h_{\ell+1} = h_\ell + g_\ell(h_\ell, Z, m)F_\ell(h_\ell), \tag{11}$$

asking whether explicit goals determine which computations should be active. This is intentionally not required by Paper 1.

14 Future PRA and Progressive Tool Integration

PRA addresses selective memory; this paper addresses goal/state control. Later typed references may represent tasks, tools, skills, documents, and observations, with progressive materialization conditioned on the active goal. Tool selection can then become goal-conditioned affordance/action selection rather than repeated schema-token processing. These are integration targets, not dependencies.

15 Feedback, RL, and Consolidation

A deployed system can accumulate tuples $(X, Z_G, Y, Z_Y, r, feedback)$ plus deterministic tests, retries, accepted edits, tool outcomes, and controller choices. This is richer experience than plain next-token text. Online base-weight learning is deferred. Later sleep/consolidation should use provenance-aware replay, adapter-only candidate updates, held-out regression gates, and rollback. RL should reward externally grounded transitions rather than blindly maximize the model's own validator.

16 Relation to Prior Work

Unified language models demonstrate shared weights under bidirectional and causal masks [2]; encoder-decoder transformers separate encoding from decoding [6, 5]; continuous prefixes condition generation on learned latent vectors [3]; self-consistency uses multiple reasoning samples [7]; and verifiers or reward models explicitly score outputs [1, 4]. The present work asks whether persistent intended task state, explicit modes, and a realized-output representation provide value beyond these components under controlled composition.

17 Success Criteria

A strong result requires more than benchmark improvement. Across multiple seeds and held-out task generators, at least several of the following should hold: bidirectional encode improves constraint fidelity under compute control; Z improves long-output retention; interventions on Z predictably alter goal satisfaction; review improves externally verified error detection; validator scores are calibrated and generalize; validation improves selection/retry at competitive compute; and internal metrics show interpretable mode/goal effects.

A strong negative result is also informative: if the causal decoder reproduces all benefits under matched resources, explicit modes or Z should not be retained merely for conceptual appeal.

18 Reproducibility Plan

Every run stores git commit, complete config, seed, dataset generator/version/hash, architecture and parameter count, optimizer/schedule, hardware, training/evaluation time, token/forward-pass counts, raw predictions, validator outputs, and plot/table source data. Headline results use multiple seeds. Raw run outputs are immutable.

The repository exposes CPU smoke and overfit gates, one-seed unstaged and staged held-out pilots, a main configuration, and a five-seed baseline suite. A timestamped run directory is created with exclusive semantics and contains the resolved configuration, provenance, materialized dataset hashes, epoch history, checkpoint, reload verification, raw JSONL predictions, and scalar metrics. Timestamped suite summaries include confidence intervals and seed-paired differences. Unit tests check generator determinism, split-family separation, multiple corruption types, counterfactual integrity, dataset-hash stability, exact parameter matching, active parameter accounting, causal prefix invariance, bidirectional suffix sensitivity, cached-encode call counts, seeded optimizer determinism, and finite multi-objective losses. Smoke and memorization-gate completion establish only that the protocol executes and can fit data; neither is entered into headline result tables.

19 Implementation Diagnostic (Not a Headline Result)

Before launching a multi-seed study, we ran a seed-0 optimization diagnostic with a small Transformer trained from scratch. A shared-example memorization gate reached 1.0 exact match for B0, B5, and B10 after 100, 250, and 250 optimizer updates, respectively; B10 also reached 1.0 validator AUROC. Thus the representative paths can fit the task and validator when generalization is deliberately removed. Table 1 instead uses disjoint example namespaces, held-out prompt topologies, and held-out corruption families. “Staged” adds 15 goal-warmup epochs and validation-selected joint training. Effects are the decrease in exact match caused by the intervention; positive values indicate degradation.

Protocol	Baseline	Exact	Facet	Zero Z_G	Shuffle Z_G	Val. AUROC
Unstaged	B0	.430	.797	—	—	—
Unstaged	B5	.109	.553	.020	.000	—
Unstaged	B10	.102	.539	-.004	.000	.476
Staged	B5	.168	.604	.090	.004	—
Staged	B10	.156	.601	.031	-.016	.575

Table 1: Seed-0 implementation diagnostic on the held-out V0 generator. This table is not a hypothesis test.

The held-out result is presently negative. Staging improves B5 and B10 relative to their unstaged variants, but both remain far below causal B0. Zeroing Z_G becomes harmful, yet shuffling it does not consistently reduce accuracy, which indicates a generic conditioning effect rather than reliable example-specific control. B10 validation rises above chance (AUROC .575; pairwise ranking accuracy .637) but is not strong enough for selection claims. These observations block a costly multi-seed headline run: the next iteration should strengthen example-specific goal conditioning and the held-out validator objective, then require shuffle and counterfactual interventions to pass preregistered gates. The exact source values and interpretation are stored in `results/pilot_iteration2.json`.

20 Persistent Goal State as a Causal Channel

The current pilot supports a narrower claim than the original aspiration. The model can be sensitive to the presence or ablation of Z_G, but this alone does not show that Z_G is an example-specific goal representation. In particular, a latent can influence generation while still acting as a generic auxiliary bias, a compressed prompt statistic, or a training-dependent shortcut. We therefore adopt a stricter operational criterion:

> A latent state represents the goal to the extent that interventions on that state, while holding relevant non-goal content fixed, cause predictable goal-consistent changes in behavior.

The next iteration therefore begins by forcing Z_G to become the only goal-information channel. Tasks are split into content C and goal specification G, the generator receives C directly, and the requested behavior must be conveyed through $Z_G = \text{Encode}(G)$. The primary controls are correct-Z, shuffled-Z, zero-Z, constant-Z, and random-Z generation. A causal gate is passed only if correct-Z materially outperforms shuffled-Z on held-out synthetic tasks. Held-out probes on frozen Z_G then separate goal decodability from goal use: high probe accuracy with weak shuffle sensitivity indicates that encoding works but conditioning fails, whereas low probe accuracy indicates that the representation itself is inadequate.

This stricter framing also motivates a refined persistence hypothesis. Persistent goal state may provide little or no benefit for short generations or shallow networks, yet become increasingly useful when intent must remain influential across many autoregressive steps or many residual transformations. The relevant signatures are therefore not limited to aggregate exact match. We will first test whether richer latent capacity and repeated per-layer access to the same latent improve correct-versus-shuffled Z_G effects in the Z-only setting, then ask whether those effects grow with generation horizon, grow with depth, or reduce late-output drift under long-horizon constraints. Negative outcomes remain informative: if Z_G stays weak even when direct goal tokens are removed, then the current representation or conditioning mechanism should be revised before larger benchmark runs.

As of Sunday, August 23, 2026, the reinjection and conditioning diagnostics show a more specific pattern than the initial negative result. Under the same small held-out Z-only strong-goal setting, naive all-layer additive reinjection of a single-vector Z_G performs worse than the input-only condition: exact match falls from .148 to .119, facet satisfaction falls from .608 to .526, the shuffled-goal effect falls from .041 to .027, and the zero-goal effect falls from .059 to .051. A periodic reinjection schedule that reintroduces the same latent every two layers partially recovers some of this loss, but still does not surpass the input-only baseline: exact match rises only to .121, facet satisfaction to .554, and the shuffled-goal effect to .031, while the zero-goal effect drops further to .020. By contrast, late-layer reinjection is the first reinjection schedule that improves on the input-only baseline: exact match rises to .164, facet satisfaction to .619, the shuffled-goal effect to .055, and counterfactual facet substitution success to .951. However, combining the two individually promising modifications—multi-vector goal state and late-layer reinjection—is sharply negative in this first pilot: exact match falls to .057, facet satisfaction to .446, the shuffled-goal effect nearly disappears at .002, and counterfactual facet substitution success drops to .746. The first non-additive conditioning diagnostic, continuous prefix conditioning with four learned latent prefix tokens, is competitive with the best additive late-layer schedule rather than clearly superior or inferior: exact match reaches .162, facet satisfaction .624, the shuffled-goal effect .057, and counterfactual facet substitution success .889. A same-day prefix-length sweep then shows that the result is not a one-setting anomaly but also not monotonic with prefix size: two prefix tokens yield exact match .156, facet satisfaction .615, and the strongest shuffled-goal effect at .063; four tokens yield exact match .162 and the strongest facet satisfaction at .624; eight tokens keep exact match at .162 but degrade facet satisfaction to .593 and reduce held-out linear-probe facet accuracy from roughly .656/.645 to .601. The two-memory prefix-KV diagnostic reaches .143 exact match, .609 facet satisfaction, and a .049 shuffled-goal effect, below both the matched two-token continuous prefix and the best late-layer additive condition despite its additional layerwise K/V parameters. The initially matched eight-epoch direct text-prefix control reaches only .102 exact match and .518 facet satisfaction, but its validation LM loss remains 1.271, revealing severe under-optimization. A 40-epoch convergence follow-up with the same direct architecture reaches .975 exact match and .995 facet satisfaction, with validation LM loss .021. Thus the apparent short-run latent advantage is an optimization-speed artifact, not evidence that latent conditioning outperforms ordinary goal tokens; the direct causal route is decisively stronger once trained to convergence in this pilot. Among latent mechanisms only, the evidence favors selective late access or a short continuous prefix over naive repeated additive access, richer latent combinations, or prefix-KV. The strongest Z-only mechanism must now receive a comparable convergence run before causal gates are judged, and prefix-KV should not be carried into larger sweeps unless depth-specific evidence later motivates revisiting it. Exact source values are stored in `results/text_prefix_convergence_pilot_iteration1.json`, `results/z_only_prefix_kv_pilot_iteration1.json`, and the preceding context, reinjection, and prefix summaries.

21 Conclusion

Encode–Generate–Review–Validate is intended as the smallest rigorous test of a broader idea: functions currently forced into homogeneous autoregressive computation may benefit from explicit mode and persistent task state. The design preserves shared weights to avoid explaining gains through parameter multiplication, uses external validation to avoid circular self-evaluation, and emphasizes causal/mechanistic analysis. If successful, it creates a controlled bridge from next-token models toward multi-timescale goal dynamics, selective computation, progressive memory, tools, and rein-

forcement learning. If unsuccessful, its ablations identify which proposed cognitive distinctions are unnecessary.

References

[1] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021. URL https://arxiv.org/abs/2110.14168.

[2] Li Dong, Nan Yang, Wenhui Wang, Furu Wei, Xiaodong Liu, Yu Wang, Jianfeng Gao, Ming Zhou, and Hsiao-Wuen Hon. Unified language model pre-training for natural language understanding and generation. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL https://proceedings.neurips.cc/paper/2019/hash/c20bb2d9a50d5ac1f713f8b34d9aac5a-Abstract.html.

[3] Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing*, pages 4582–4597, 2021. doi: 10.18653/v1/2021.acl-long.353. URL https://aclanthology.org/2021.acl-long.353/.

[4] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/b1efde53be364a73914f58805a001731-Abstract.html.

[5] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020. URL http://jmlr.org/papers/v21/20-074.html.

[6] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[7] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=1PL1NIMMrw.